

Функциональные возможности Python: lambda-функции, map(), filter()

1. Функциональное программирование в Python

Python поддерживает элементы функционального программирования, где функции являются первоклассными объектами. Это означает, что:

- Функции можно передавать как аргументы.
- Функции можно возвращать из других функций.
- Можно использовать анонимные функции (**lambda**).
- Встроенные функции **map()** и **filter()** позволяют работать с коллекциями без циклов.

2. Lambda-функции (анонимные функции)

lambda - это анонимная функция, записанная в одну строку.

Она принимает любое количество аргументов, но возвращает только одно выражение.

Синтаксис:

```
lambda: аргументы: выражение
```

Пример: сложение двух чисел

```
add = lambda x, y: x + y
print(add(3, 5)) # 8
```

Пример: возведение числа в квадрат

```
square = lambda x: x ** 2
print(square(4)) # 16
```

Lambda-функция внутри **print()**

```
print(
    (lambda x: x * 2)(6)
)
```

Когда использовать **lambda**?

- ✓ Когда нужна короткая функция для одноразового использования.
- ✓ Когда передаём функцию в **map()** или **filter()**.
- ✓ Когда функция содержит одно выражение и не требует **return**.
- ✗ Когда НЕ стоит использовать **lambda**?
 - Если код становится нечитаемым.

- Если логика сложная (лучше использовать `def`).
Обычная функция vs `lambda`

```
# Обычная функция
def multiply(x, y):
    return x * y

# lambda-функция
multiply_lambda = lambda x, y: x * y

print(multiply(2, 3)) # 6
print(multiply_lambda (2, 3)) # 6
```

3. Функция `map()`

Функция `map()` применяет переданную функцию ко всем элементам последовательности.

Она заменяет циклы `for`, делая код компактнее и понятнее.

Синтаксис:

```
map(функция, итерируемый объект)
```

Пример 1: Возведение чисел в квадрат

```
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x ** 2, numbers))
print(squared) # [1, 4, 9, 16, 25]
```

Пример 2: Применение `map()` к строкам

```
words = ["hello", "world", "python"]
upper_words = list(map(str.upper, words))
print(upper_words) # ['HELLO', 'WORLD', 'PYTHON']
```

Пример 3: Преобразование температур

Переведём список температур из градусов Цельсия в Фаренгейты:

Формула: $F = C * 9/5 + 32$

```
celsius = [0, 10, 20, 30, 40]
fahrenheit = list(map(lambda c: c * 9/5 + 32, celsius))
print(fahrenheit) # [32.0, 50.0, 68.0, 86.0, 104.0]
```

Когда использовать `map()`?

- ✓ Когда нужно одинаково преобразовать все элементы списка.
- ✓ Когда можно заменить цикл `for` на короткую функцию.

4. Функция `filter()`

Функция `filter()` отбирает элементы, оставляя только те, для которых переданная функция возвращает `True`.

В отличие от `map()`, `filter()` не изменяет элементы, а только выбирает нужные.

Синтаксис:

```
filter(функция условие, итерируемый объект)
```

Пример 1: Фильтрация чётных чисел

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers) # [2, 4, 6]
```

Пример 2: Отбор строк по длине

```
words = ['hi', 'hello', 'world', 'Python']
long_words = list(filter(lambda w: len(w) > 3, words))
print(long_words) # ['hello', 'world', 'Python']
```

Пример 3: Фильтрация положительных чисел

```
numbers = [-10, -5, 0, 5, 10, 15]
positive_numbers = list(filter(lambda x: x > 0, numbers))
print(positive_numbers) # [5, 10, 15]
```

Когда использовать `filter()`?

- ✓ Когда нужно отобрать элементы списка по условию.
- ✓ Когда можно заменить `for + if` на более компактную запись.

5. Сравнение `map()` и `filter()`

Функция	Назначение	Возвращает
<code>map()</code>	Преобразует каждый элемент	Новый список
<code>filter()</code>	Отбирает элементы по условию	Новый список (только True)

Пример: `map()` и `filter()` вместе

```
numbers = [1, 2, 3, 4, 5, 6]

# map: удваиваем числа
doubled = list(map(lambda x: x * 2, numbers))
print(doubled) # [2, 4, 6, 8, 10, 12]

# filter: оставляем только чётные
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers) # [2, 4, 6]

# Комбинируем map() и filter()
filtered_and_doubled = list(map(lambda x: x * 2, filter(lambda x:
x % 2 == 0, numbers)))
print(filtered_and_doubled) # [4, 8, 12]
```

6. Итоги

- ✓ Lambda позволяет записывать анонимные функции в одну строку.
- ✓ `map()` применяет функцию ко всем элементам списка.
- ✓ `filter()` выбирает только нужные элементы.
- ✓ Эти функции делают код компактнее и понятнее, но иногда циклы `for` более читаемы.

7. Практические задания

1. Используя `map()`, переведите список слов в список их длин.
2. Используя `filter()`, отберите из списка только числа больше 10.
3. Напишите `lambda`-функцию, которая проверяет, является ли строка палиндромом.

4. Используйте `map()` и `filter()` вместе, чтобы получить список квадратов чётных чисел.
5. Используя `filter()`, отберите из списка пустые строки.

Вывод

Функции `lambda`, `map()` и `filter()` помогают писать более короткий, функциональный и читаемый код. Они широко используются в обработке данных, анализе текстов, работе с числами и оптимизации операций.